

Arquitectura, flujo y ejecución del proyecto fibonacci_photonics 2.0

Emanuel Orozco Gallego

28 de septiembre de 2026

Resumen

Manual de arquitectura del repositorio que reproduce Reyes-Gómez *et al.*, Phys. Rev. B **81**, 153101 (2010). Describe cómo está organizado el código, cómo circulan los datos desde un YAML hasta una figura, qué dependencias hacen falta y cómo instalar y ejecutar el proyecto de punta a punta. La física está en *Guía conceptual*; el algoritmo TMM, en *Solución numérica e implementación*; el método de ondas planas, en *El método de expansión en ondas planas*.

Índice

1. Qué es este repositorio	2
2. Árbol del proyecto	2
3. Arquitectura de la biblioteca	3
3.1. Capas y dependencias	3
3.2. El contrato DispersionSolver	3
3.3. Validaciones	3
3.4. Módulos y responsabilidades	4
3.5. Objetos que cruzan las fronteras	4
3.6. API pública	4
4. Capa de configuración	5
5. Reproducción, estudios y scripts	5
6. Flujo de una corrida	5
7. Pruebas	6
8. Dependencias	7
8.1. Tiempo de ejecución (Python)	7
8.2. Sistema, para los PDFs	7
9. Cómo ejecutar el proyecto	7
9.1. 1. Entorno	7
9.2. 2. Pruebas y lint	8
9.3. 3. Figuras	8
9.4. 4. Documentos PDF	8
9.5. 5. Todo de una vez	8
9.6. Cambiar un parámetro	8
9.7. Limpieza	8

10.Problemas frecuentes	8
11.Dónde está cada documento	9
12.Resumen operativo	9

1. Qué es este repositorio

Es una reimplementación en Python del Brief Report de 2010 con dos métodos independientes: la matriz de transferencia (TMM), que es el método de producción, y la expansión en ondas planas (PWE), que la verifica. No hay servidor web ni interfaz gráfica: el producto son *datos*, *figuras* y *PDFs*, separados por método en `results/tmm/`, `results/pwe/` y `results/comparison/`.

Tres capas, de dentro hacia afuera:

1. **Biblioteca** `src/fibonacci_photonics/`: física, métodos numéricos, post-proceso, configuración tipada y la reproducción de cada figura.
2. **Scripts** `scripts/`: envoltorios de pocas líneas que llaman a la biblioteca; el punto de entrada único es `scripts/run_figure.py` (o el comando `fibonacci-figure`).
3. **Documentos** `docs/`: LaTeX que incrusta las figuras generadas.

Los parámetros científicos no viven en el código: viven en `configs/figure_ON.yaml` y se validan con Pydantic al cargarlos. Cambiar un plasma o un ángulo no exige editar Python, y una errata en un YAML falla al arrancar, no a mitad de una corrida de horas.

2. Árbol del proyecto

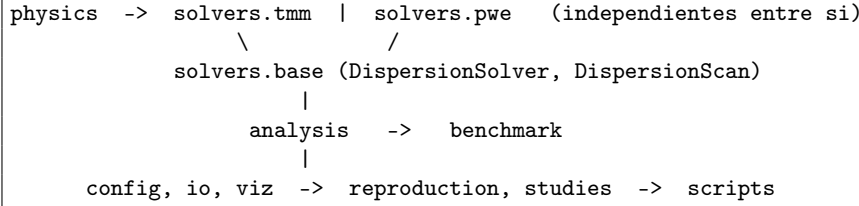
<code>analisis_numerico/</code>	
<code>src/fibonacci_photonics/</code>	biblioteca (layout src, con <code>py.typed</code>)
<code>physics/</code>	constantes, unidades, materiales, TE/TM, Fibonacci, SuperlatticeSpec, medio efectivo
<code>solvers/</code>	contrato DispersionSolver y DispersionScan
<code>tmm/</code>	matriz de transferencia y TMMSolver
<code>pwe/</code>	Fourier, QEP de Bloch, forma $w(k)$, PWESolver, convergencia en armónicos
<code>analysis/</code>	raíces, bordes de banda, cierre de bandas, subbandas plasmonicas, anchos vs theta
<code>config/</code>	esquemas Pydantic v2 y cargador de YAML
<code>benchmark/</code>	comparacion TMM vs PWE, tiempos, informes
<code>reproduction/</code>	Figs. 1-6 con cualquiera de los dos metodos + CLI
<code>studies/</code>	convergencia TMM, benchmark del libro, estudios PWE
<code>io/</code>	rutras, JSON/npz, provenance, tablas LaTeX
<code>viz/</code>	estilo PRB y curvas de dispersion
<code>errors.py</code>	InvalidParameterError, ConvergenceWarning
<code>parallel.py</code>	procesos con un hilo de BLAS
<code>tests/unit/</code>	contrato, validaciones, modulos
<code>tests/analytic/</code>	soluciones cerradas
<code>tests/regression/</code>	valores de referencia y conteos del paper
<code>configs/figure_ON.yaml</code>	fisica y mallas de cada figura
<code>configs/pwe/</code>	numerica del PWE y estudios
<code>configs/comparison/</code>	estudio de eficiencia
<code>scripts/run_figure.py</code>	runner unico (<code>--method tmm pwe both</code>)
<code>scripts/{tmm,pwe}/</code>	envoltorios por figura y estudio
<code>scripts/comparison/</code>	envoltorios de benchmark/
<code>scripts/verify_results.py</code>	verificacion automatica (make verify)
<code>results/<metodo>/<estudio>/</code>	data/, output/, README.md
<code>docs/</code>	auditoria, papers, guias, este documento
<code>presentacion/</code>	presentacion (make presentation)
<code>notebooks/</code>	exploracion
<code>pyproject.toml</code>	paquete, dependencias, pytest, ruff, mypy
<code>requirements.txt</code>	-e <code>.[dev]</code>
<code>Makefile</code>	tests, lint, resultados, PDFs

El entorno virtual `.venv/` y las cachés no forman parte del código fuente. `.gitignore` también excluye auxiliares de LaTeX y los recortes del PDF original.

3. Arquitectura de la biblioteca

3.1. Capas y dependencias

La dependencia es acíclica y va de lo más físico a lo más aplicado:



`solvers.tmm` y `solvers.pwe` no se importan entre sí. El post-proceso (`analysis`) y la comparación (`benchmark`) se escriben contra el contrato `DispersionSolver`, así que no saben qué método produjo los datos. Solo `viz`, `reproduction`, `studies` y `benchmark` importan Matplotlib: el núcleo numérico se usa sin pintar nada.

3.2. El contrato `DispersionSolver`

Un método numérico es cualquier objeto con

- `method` ("tmm" o "pwe") y `spec`;
- `semitrace(m, theta, nu_ghz, polarization)`: $R_m(\nu)$ vectorizada, con la forma de `nu_ghz`;
- `scan(m, theta, nu_ghz, polarization)`: un `DispersionScan` en una malla creciente;
- `evaluator(m, theta, polarization)`: $\text{Re } R_m(\nu)$ escalar con contador de llamadas, para Brent.

`TMMSolver(spec, algorithm="recurrence"|"product")` y `PWESolver(spec, harmonics_per_layer, rule, workers)` lo implementan; `get_solver("tmm"|"pwe", spec, ...)` los construye por nombre. Ambos son *dataclasses* inmutables. `PWESolver` añade `n_max(m)`, `refined(factor)` (truncamiento de control) y `convergence_report`, que compara con 1.5 veces más ondas planas y emite `ConvergenceWarning` si R cambia en las bandas o alguna frecuencia pasa de banda a gap.

3.3. Validaciones

Todas las entradas se validan al entrar al método y fallan con `InvalidParameterError` (subclase de `ValueError`):

- frecuencias finitas y > 0 (el Drude diverge en $\nu = 0$); en `scan`, además, malla 1-D estrictamente creciente;
- ángulo $\theta \in [0, \pi/2]$ y orden de Fibonacci entero ≥ 0 ;
- espesores y c positivos, $\varepsilon_A, \mu_A \neq 0$, plasmas ≥ 0 y fondos > 0 al construir la superred;
- armónicos por capa, procesos y factor de truncamiento positivos.

3.4. Módulos y responsabilidades

Módulo	Responsabilidad
physics/	c , GHz, mm; aire y Drude $\varepsilon(\omega), \mu(\omega)$; q , Q , χ TE/TM; S_m , F_m , L_m ; SuperlatticeSpec ; $\langle \varepsilon \rangle$, $\langle \mu \rangle$
solvers/base.py, scan.py solvers/tmm/	contrato, validaciones, DispersionScan , máscara de banda, kL_m/π matrices de capa, T_m , R_0, R_1, R_2 analíticas, recurrencia de trazas, TMMSolver
solvers/pwe/	coeficientes de la celda y Toeplitz; QEP en k con regla de Li; forma $\omega(k)$ del libro y de Drude; SemitraceEvaluator ; PWESolver ; convergencia
analysis/roots.py analysis/band_edges.py	Brent acotado bandas y gaps más estrechos que la malla, validación con dos truncamientos, malla logarítmica hacia ν_m
analysis/band_closure.py analysis/plasmon_modes.py, plasmon_bands.py analysis/bandwidth.py	bordes $ R = 1$, puntas y gaps ocultos que cierran las curvas $\nu(k)$ conteo de subbandas F_{m-2} ; estrategia densa (TMM) y adaptativa validada (PWE)
config/	$\nu_{\min}, \nu_{\max}$ frente a θ (Fig. 6)
benchmark/	esquemas Pydantic y load_figure , load_pwe_numerics , load_spec compare_scans , emparejamiento de bandas, tiempos, informes de las Figs. 1–6, eficiencia y resumen
reproduction/ studies/	FigureRun , las seis figuras para ambos métodos, run_figure y la CLI convergencia TMM de la Fig. 4, celda de Fourier, benchmark del libro, contaminación espectral, convergencia y costo del PWE
io/, viz/	rutas de results/ , JSON, npz, metadatos, tablas LaTeX; estilo PRB, curvas y guardado

3.5. Objetos que cruzan las fronteras

- **SuperlatticeSpec**: espesores en metros y Drude en rad/s. Lo construye **SuperlatticeConfig.to_spec()** a partir del YAML.
- **DispersionScan**: ν , R_m , máscara de banda y kL_m/π . Lo produce **solver.scan**.
- **FrequencyInterval**: un trozo conexo $[\nu_{\min}, \nu_{\max}]$.
- **BandEdges**: bordes y puntas refinados de un barrido.
- **BandwidthPoint**: anchos y centros a un θ dado (Fig. 6).

3.6. API pública

```
import numpy as np
from fibonacci_photonics import PWESolver, TMMSolver, load_spec

spec = load_spec("figure_03.yaml")
nu = np.linspace(0.5, 5.0, 2001)
tmm = TMMSolver(spec).scan(4, np.pi / 6, nu, "TE")
pwe = PWESolver(spec, harmonics_per_layer=16)
report = pwe.convergence_report(4, np.pi / 6, nu[::200])
```

4. Capa de configuración

Archivo	Uso
<code>configs/figure_01.yaml</code>	$\nu_e = \nu_m = 3$ GHz, $m = 3, 4$, cuatro θ
<code>configs/figure_02.yaml</code>	zoom 2–4 GHz, $m = 3 \dots 6$
<code>configs/figure_03.yaml</code>	como Fig. 1 con $\nu_m = 1$ GHz
<code>configs/figure_04.yaml</code>	ventanas densas cerca de 1 GHz
<code>configs/figure_05.yaml</code>	palos vs $m = 2 \dots 8$
<code>configs/figure_06.yaml</code>	$\nu(\theta)$ relleno, $m = 2 \dots 7$ (inferida)
<code>configs/pwe/figure_0N.yaml</code>	armónicos por capa, mallas y órdenes del PWE
<code>configs/pwe/book_benchmark.yaml</code>	crystal $\varepsilon_1 = 1$, $\varepsilon_2 = 9$ del libro
<code>configs/pwe/convergence.yaml</code>	estudios de convergencia y costo
<code>configs/pwe/spectral_pollution.yaml</code>	forma $\omega(k)$ con Drude
<code>configs/comparison/efficiency.yaml</code>	estudio de tiempos TMM vs PWE en un núcleo

Cada archivo tiene su esquema en `config/schema.py` (`DispersionConfig`, `ModeCountConfig`, `WindowedConfig`, `AngleSweepConfig`, las numéricas del PWE y los estudios). Los modelos son inmutables y rechazan claves desconocidas (`extra="forbid"`). Además de tipos y signos, se validan relaciones: ventanas con $\nu_{\min} < \nu_{\max}$, una etiqueta por ángulo, una ventana por etiqueta, $\theta \leq \pi/2$, mallas de al menos dos puntos y que los armónicos por orden cubran los órdenes pedidos. Un error se informa como `ConfigError` con el archivo y el campo culpables.

La física de cada figura está en un solo sitio (`configs/figure_0N.yaml`) y la leen los dos métodos; `configs/pwe/` solo añade parámetros numéricos del PWE. `load_spec()` sin argumentos devuelve la superred del artículo. Campo `source: paper` si el original da los números; `inferred_from_figures_3_to_5` en la Fig. 6.

5. Reproducción, estudios y scripts

`reproduction.run_figure(figure_id, method)` abre un `FigureRun` (configuración validada, numérica del PWE si aplica, directorios de salida y procesos), construye el solver con `get_solver`, calcula, dibuja y termina con `finish`, que escribe la figura, `summary.json`, `metadata.json` y el `README.md` de la carpeta. Las seis figuras comparten el mismo código para los dos métodos; solo difieren en la estrategia de subbandas plasmónicas (densa en la TMM, adaptativa y validada con 1.5 veces más ondas planas en el PWE).

```
python scripts/run_figure.py figure_01 --method tmm
python scripts/run_figure.py 2 4 --method pwe --workers 8
fibonacci-figure all --method both # tras pip install -e .
```

Los scripts por figura (`scripts/tmm/figure_0N.py`, `scripts/pwe/figure_0N.py`) son envoltorios de una línea sobre `run_figure`; los estudios (`scripts/tmm/convergence.py`, `scripts/pwe/{book_benchmark,cell_fourier,spectral_pollution,convergence}.py`) llaman a `fibonacci_photonics.studies`; y `scripts/comparison/*.py` llaman a `fibonacci_photonics.benchmark`. `efficiency.py` fija un hilo de BLAS antes de importar NumPy para que los tiempos sean comparables; `-plot-only` redibuja sin medir.

Cada carpeta tiene un `run_all.py` que lanza sus scripts en orden con el mismo intérprete y aborta si uno falla. Los envoltorios antiguos `scripts/reproduce_figure_0N.py` siguen ejecutando la versión TMM. `PWE_WORKERS=n` limita los procesos del PWE.

6. Flujo de una corrida

```

configs/figure_ON.yaml (+ configs/pwe/figure_ON.yaml)
|
v
load_figure / load_pwe_numerics      (Pydantic: tipos, rangos, relaciones)
|
v
SuperlatticeSpec -> get_solver("tmm" | "pwe", spec, ...)
|
+--> solver.scan(m, theta, nu, pol)      -> DispersionScan
+--> close_band_edges(solver, scan)      (bordes y puntas exactos)
+--> dense_grid_bands / adaptive_bands  (Figs. 2, 4, 5)
+--> bandwidth_versus_angle             (Fig. 6)
|
v
FigureRun.finish
+--> results/<metodo>/figure_ON/output/figure_ON.{pdf,png[,svg]}
+--> results/<metodo>/figure_ON/data/*.npz, summary.json, metadata.json

```

Los PDFs científicos no recalculan física: incluyen las figuras ya generadas en **results/**. Por eso hay que generar los resultados *antes* de compilar los documentos si se cambió el código.

Flujo del Makefile:

```

make all
make tests      -> pytest tests
make results    -> make tmm pwe comparison
  make tmm      -> scripts/tmm/run_all.py (Figs. 1-6 + convergencia)
  make pwe      -> scripts/pwe/run_all.py (~50 min con 16 procesos)
  make comparison -> scripts/comparison/run_all.py (incluye eficiencia)
make documents  -> pwe-docs, guides, paper, docs
make verify     -> scripts/verify_results.py
make lint       -> ruff check + mypy
make reproduce  -> results + documents + verify (sin tests)
make presentation -> presentacion/ (objetivo aparte)

```

Todos los PDF se compilan con una sola regla de patrón (dos pases de **pdflatex**, que se detiene en el primer error) y quedan junto a su fuente en **docs/<documento>/**. **make verify** comprueba que existan todas las salidas, que se cumplan los criterios de acuerdo TMM vs PWE (conteos F_{m-2} , errores de semitraza, de borde y de ancho), que cada tabla y figura citada exista y que cada PDF esté al día y compile limpio. Con **-reference DIR** compara además los datos con una corrida anterior.

7. Pruebas

pyproject.toml configura **testpaths = ["tests"]** y **pythonpath = ["src"]**; no hace falta instalar el paquete. La batería tiene unas 320 pruebas y tarda del orden de 10 s.

Carpeta	Qué garantiza
<code>unit/</code>	contrato <code>DispersionSolver</code> para ambos métodos; rechazo de frecuencias ≤ 0 , no finitas o mal ordenadas, ángulos y órdenes inválidos; numérica del PWE; <code>ConvergenceWarning</code> ; esquemas Pydantic y YAML inválidos; Fourier, matrices de transferencia, PWE frente a TMM, bordes y análisis de bandas
<code>analytic/</code>	límite homogéneo $R_m = \cos(Q_A L_m)$ en TMM y PWE; bicapa de cuarto de onda $R = -5/3$; $\det T = 1$; invariante de Kohmoto; TE = TM en $\theta = 0$ y dualidad $\varepsilon \leftrightarrow \mu$; Parseval; razón áurea; frecuencia de $\langle n \rangle = 0$
<code>regression/</code>	R_m , bordes y subbandas iguales a <code>reference_values.json</code> (versión previa a la refactorización); $N_{\text{modos}} = F_{m-2}$ en las ventanas de las Figs. 2, 4, 5 y 6

`confctest.py` fija un hilo de BLAS y expone dos superredes: plasmas iguales (3,3 GHz, Figs. 1–2) y partidos (3,1 GHz, Figs. 3–6). Tres casos se omiten cuando una identidad de Fibonacci pide $k > m$.

`make lint` ejecuta Ruff (estilo, imports, zip estricto, reglas de NumPy) y mypy sobre todo el paquete, que declara sus tipos con `py.typed`.

8. Dependencias

8.1. Tiempo de ejecución (Python)

`pyproject.toml` es la fuente única; `requirements.txt` solo instala el paquete editable con el extra `dev`. Python ≥ 3.10 ; el desarrollo se hizo con Python 3.14.

Paquete	Mínimo	Para qué
<code>numpy</code>	1.24	arrays, trigonométricas complejas
<code>scipy</code>	1.10	Brent, autovalores del PWE
<code>matplotlib</code>	3.7	figuras
<code>pyyaml</code>	6.0	configs
<code>pydantic</code>	2.5	validación de configs
<code>pytest, ruff, mypy, types-PyYAML</code>	—	extra dev

No hay CUDA, MPI ni bases de datos. No se descarga nada en tiempo de cálculo.

8.2. Sistema, para los PDFs

`make` y `pdflatex` (TeX Live) con `babel-spanish`, `amsmath`, `graphicx`, `booktabs`, `hyperref`, `listings`, `geometry`, `siunitx` y `enumitem`. Sin LaTeX se pueden ejecutar pruebas y figuras; solo fallan los objetivos de documentos.

9. Cómo ejecutar el proyecto

9.1. 1. Entorno

```
cd analisis_numerico
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt      # = pip install -e ".[dev]"
```

9.2. 2. Pruebas y lint

```
make tests
make lint
```

9.3. 3. Figuras

```
python scripts/run_figure.py all --method tmm      # segundos
python scripts/run_figure.py 4 --method tmm        # una sola
PWE_WORKERS=8 python scripts/pwe/run_all.py        # PWE y sus estudios
python scripts/comparison/run_all.py               # requiere los dos
make results                                       # los tres
```

9.4. 4. Documentos PDF

```
make documents # todos
make verify    # comprobacion final
```

9.5. 5. Todo de una vez

```
make all
```

Orden interno: pruebas → resultados (TMM, PWE, comparación) → documentos → verificación.

9.6. Cambiar un parámetro

Edite el YAML (por ejemplo `omega_m_over_2pi_ghz` en `configs/figure_03.yaml`) y ejecute solo esa figura. Si el valor no es válido, la carga se detiene con un `ConfigError` que indica el campo. Para TM: `polarization: TM`.

9.7. Limpieza

`make clean` borra los auxiliares de LaTeX; no toca figuras ni el entorno virtual.

10. Problemas frecuentes

- `ModuleNotFoundError: fibonacci_photonics`: instale el paquete (`pip install -e .`) o use `PYTHONPATH=src`.
- `InvalidParameterError`: las frecuencias deben ser > 0 : el Drude diverge en $\nu = 0$; la malla no debe incluir el origen.
- `ConvergenceWarning`: el PWE no convergió con los armónicos pedidos (típicamente pegado a ν_m); aumente `harmonics_per_layer` o use la TMM.
- Fig. 1 sin gaps a $\theta = 0$ y $\omega_e = \omega_m$: no es un bug, es adaptación de impedancia.
- El PWE satura la máquina: `PWE_WORKERS=n`.
- El Makefile apunta a `.venv/bin/python`; si el venv está en otra ruta, use el intérprete explícito.

11. Dónde está cada documento

PDF en docs/	Contenido
<code>paper_reimplementation.pdf</code>	artículo de la reproducción
<code>software_documentation.pdf</code>	manual corto de la API
<code>guia_conceptual.pdf</code>	física desde cero
<code>solucion_numerica.pdf</code>	TMM y código de las figuras
<code>arquitectura.pdf</code>	este documento
<code>metodo_ondas_planas.pdf</code>	formulación y resultados del PWE
<code>comparacion_tmm_pwe.pdf</code>	anexo de comparación TMM vs PWE

Fuentes LaTeX en `docs/`. Auditoría previa al código: `docs/fase_A_auditoria_matematica.md`. Informe de validación: `docs/INFORME_VALIDACION.md`. El paper original y el material del PWE (capítulo del libro, cuaderno y PWE_2D.m) se guardan solo en local, en `referencias/`.

12. Resumen operativo

```
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
make tests lint
python scripts/run_figure.py all --method tmm
make documents verify
```